课程笔记

CS336 第 3 讲：现代 Transformer 架构与超参数共识

Codex 基于 Stanford CS336 2025 第 3 讲视频、字幕与官方课程材料重写

2026-04-07

视频作者/频道：Stanford Online
发布日期：2025 年 04 月 16 日
视频时长：1:27:03
视频链接：`https://www.youtube.com/watch?v=ptFiH_bHnJw`

目录

1 这讲到底在回答什么问题 **2**

1.1 这讲为什么不追求“完美理论解释” 2

1.2 本章小结 3

2 现代 Transformer 的共同骨架：LLaMA-like 不是口号，而是收敛方向 **3**

2.1 从原始 Transformer 到“简单现代变体” 3

2.2 为什么课程说当代模型整体在向 LLaMA-like 收敛 3

2.3 本章小结 3

3 Norm 与残差流：为什么几乎所有现代 LM 都偏向 pre-norm **3**

3.1 pre-norm vs post-norm：这可能是当代最强共识之一 3

3.2 为什么 pre-norm 会表现得更稳 4

3.3 LayerNorm vs RMSNorm：现代模型为什么偏向后者 5

3.4 更进一步：为什么现代模型还喜欢顺手去掉 bias 6

3.5 本章小结 6

4 FFN 与激活函数：为什么现代模型偏爱 *GLU **6**

4.1 ReLU、GeLU、SwiGLU 不是同一层面的选择 6

4.2 Gated linear units 为什么会被认为“通常更好” 7

4.3 为什么 gated FFN 的维度比例会变 7

4.4 本章小结 7

5 Position Embeddings、超参数与稳定性：哪些是共识，哪些只是经验盆地 **8**

5.1 位置编码为什么最终大多又收敛到 RoPE 8

5.2 Feedforward ratio、head dim、aspect ratio：为什么很多超参数 surprisingly boring . . 8

5.3 regularization 与 weight decay：大模型真的还需要正则吗 9

5.4 稳定性技巧：z-loss、QK norm 与 soft-capping 9

5.5 GQA / MQA：为什么这讲最后会碰到 attention head 成本 9

5.6 本章小结 10

6 总结与延伸 **10**

1 这讲到底在回答什么问题

第 2 讲把语言模型训练压缩成了一套资源会计语言：参数、梯度、优化器状态、显存、FLOPs、MFU。第 3 讲则向前一步，开始回答“既然我们已经知道训练很贵，那么模型骨架应该长什么样？”这讲并不打算重新推导原始 Transformer，而是聚焦于一个更贴近真实工业系统的问题：**2024–2025 年主流大语言模型，在架构与训练超参数上究竟形成了哪些相对稳定的共识？**

课程一开始就把主题说得非常直白：今天最好的学习方式仍然是 hands-on experience，第二好的方式则是“尽量从别人已经付过的试错成本里学习”。这句话实际上给整讲定下了方法论基调。它不是要求你盲从某个单一配方，而是要求你去总结各家公开模型在长期试验后形成的收敛趋势。

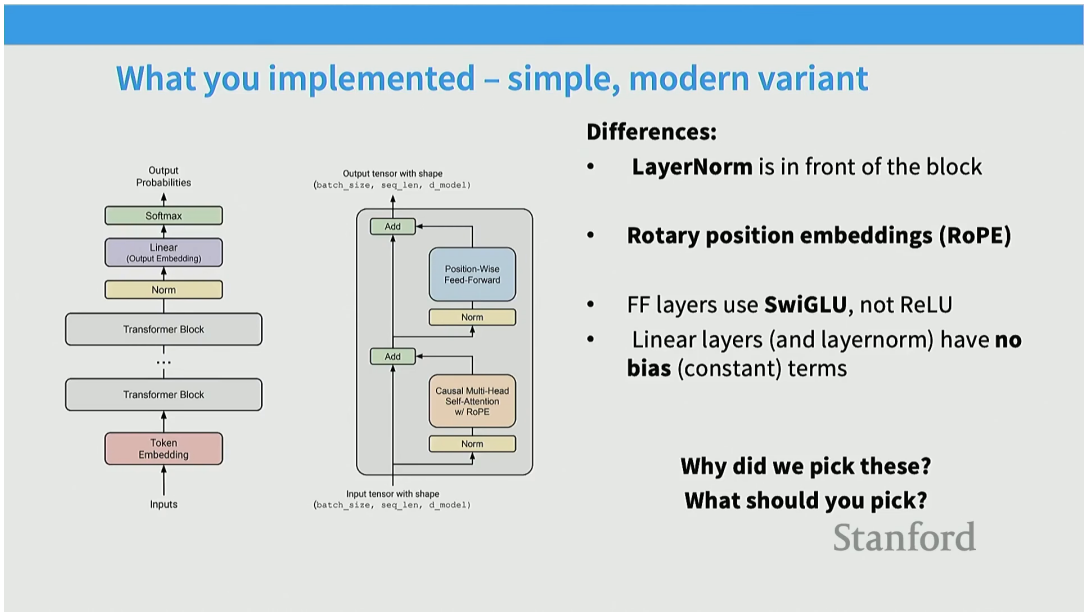

图 1：课程先把“你在作业里实现的简单现代变体”摆在台面上，再问：这些选择为什么是现代共识？[1]

这也是第 3 讲特别重要的原因：如果说第 2 讲帮你学会了资源账本，那么第 3 讲就是在教你读架构账本。换句话说，它在训练你区分“哪些改动是审美偏好，哪些改动是经验收敛，哪些改动背后其实有很现实的系统与优化动机”。

1.1 这讲为什么不追求“完美理论解释”

课程对现代架构的态度非常务实。它承认很多设计并不是从一套统一理论里自然推导出来的，而是大量模型发布、训练经验与 paper ablation 共同塑造出来的。这意味着读者需要接受一个现实：**现代大模型架构不是一套封闭、完美、不可更改的真理体系，而是一个高度经验驱动、但又逐渐收敛出稳定模式的工程空间。**

第 3 讲的核心任务

不是重新发明 Transformer，而是学会识别“哪些架构和超参数选择在真实大模型里反复出现，因此值得当成默认起点”。

[1] 视频画面时间区间：00:03:00–00:03:20。该图来自课程视频中的屏幕画面。

1.2 本章小结

第 3 讲的入口非常清晰：在理解资源约束之后，我们终于开始问“该造什么模型”。它要教的不是某个孤立 trick，而是如何从众多公开模型中提炼架构与超参数的共识部分。

2 现代 Transformer 的共同骨架：LLaMA-like 不是口号，而是收敛方向

2.1 从原始 Transformer 到“简单现代变体”

课程先快速回顾了原始 Transformer 的几个关键选择：位置编码用正弦余弦，FFN 用 ReLU，norm 是 post-norm 的 LayerNorm。随后马上给出“what you implemented – simple, modern variant”这张对照页，把当前更常见的选择并排摆出来：

- LayerNorm 挪到 block 前面，即 pre-norm。
- 位置编码换成 RoPE。
- FF 层改用 SwiGLU 而非 ReLU。
- 线性层和 norm 层里的 bias 往往被拿掉。

这张对照页很有价值，因为它把“现代模型和教科书 Transformer 的差别”压缩到了几个最实用的维度。课程真正关心的不是这些改动是否华丽，而是：**为什么多数现代模型会朝这个方向聚拢？**

2.2 为什么课程说当代模型整体在向 LLaMA-like 收敛

第 3 讲最重要的整体判断之一，是“架构共识并不算高，但存在明显向 LLaMA-like 靠拢的趋势”。这句话的意思不是说所有模型都一模一样，而是说：在 norm 位置、激活函数、位置编码和 bias 等局部设计上，越来越多模型选择了相似方案。

如何理解“LLaMA-like”

它不意味着所有模型都抄 LLaMA，而是意味着在高参数量、长训练周期和工业系统约束下，一些看似小的架构细节已经逐渐表现出更好的综合性价比，因此开始形成默认配置。

这种收敛本身就很值得重视。因为在大量模型并行试错之后还能幸存下来的设计，通常都不只是“论文里看起来不错”，而是在训练稳定性、实现复杂度、性能与系统效率之间达成了比较稳的折中。

2.3 本章小结

现代 Transformer 并没有完全统一标准答案，但已经出现了明显的工程共识区。pre-norm、RoPE、SwiGLU、RMSNorm 和去 bias 化，正是这条收敛路线里最核心的标志。

3 Norm 与残差流：为什么几乎所有现代 LM 都偏向 pre-norm

3.1 pre-norm vs post-norm：这可能是当代最强共识之一

课程在 norm 这一块的态度非常鲜明：**“one thing everyone agrees on” 基本就是 pre-norm**。这几乎是现代架构里最稳定的共识之一。原始 Transformer 使用的是 post-norm，也就是每个子层加

回残差后再做 LayerNorm；而现代做法更倾向在进入子层前先做 norm，让主残差流尽量保持直通特性。

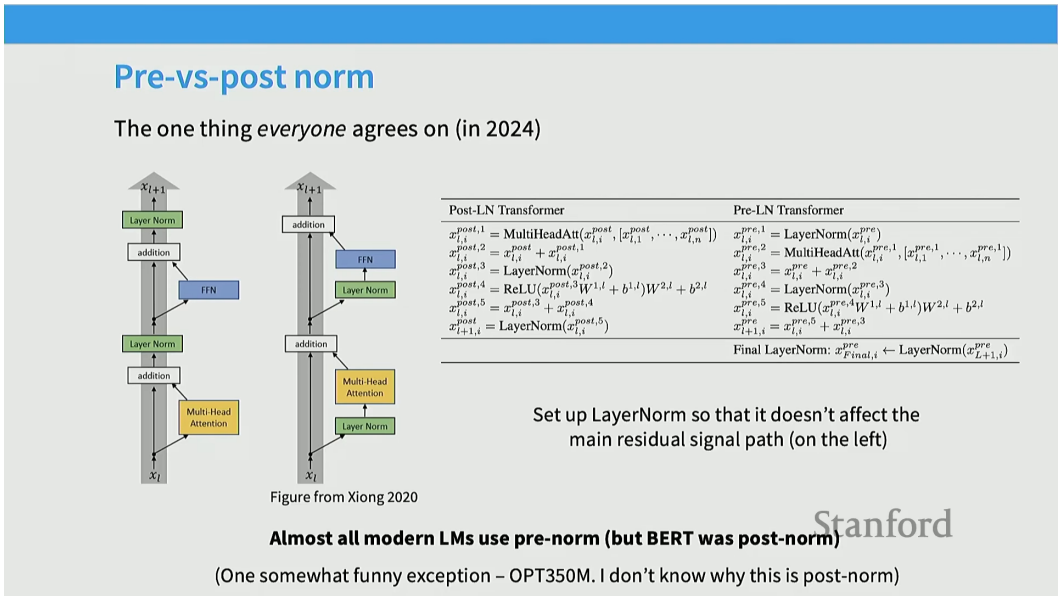

图 2: 课程用 pre-norm / post-norm 并列图说明：关键不是“norm 在哪”，而是 residual signal path 是否被过度扰动。[2]

这背后的核心直觉是：残差连接之所以宝贵，是因为它提供了一条相对稳定的信息与梯度通道。如果每次回到主干都立刻做强归一化，残差的“直通性”就会被削弱。pre-norm 的好处，是把归一化放到分支里，尽量不干扰主残差流。

3.2 为什么 pre-norm 会表现得更稳

课程回顾的解释主要有两类。第一类是梯度衰减与传播问题：post-norm 更容易让深层网络里的梯度行为变差；第二类是训练中的梯度 spike 与大学习率不稳定。它给出的总结非常符合工程经验：**pre-norm 往往带来更稳定的优化、更好的梯度传播，也更容易支撑较大的学习率和更深的网络**。

[2] 视频画面时间区间：00:06:09–00:07:45。该图来自课程视频中的屏幕画面。

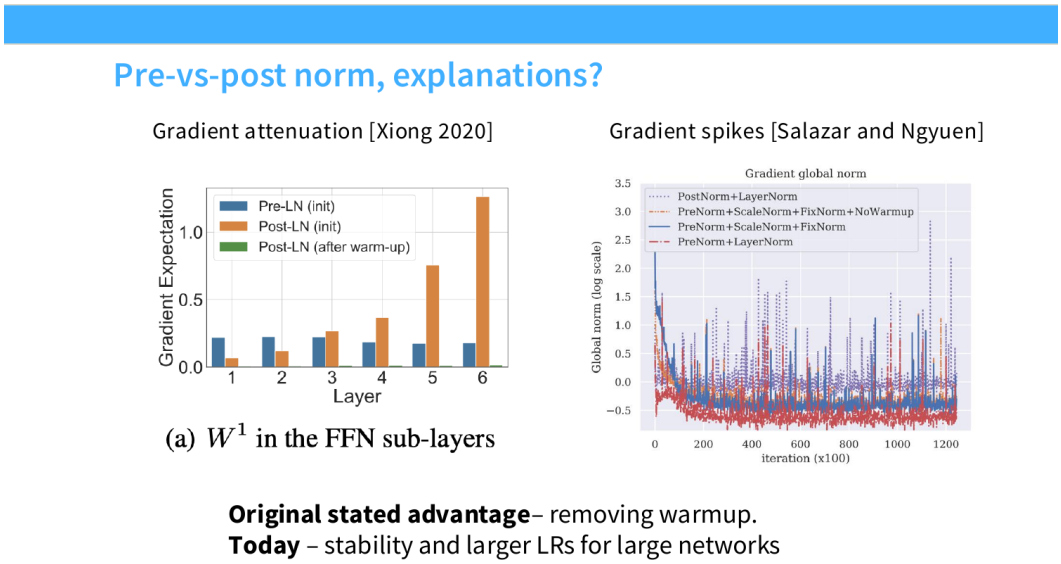

图 3: 讲义里用两个实证图解释 pre-norm 相对 post-norm 的稳定性优势：左边是梯度衰减，右边是梯度 spike。

课程还指出，早期很多论文喜欢把 pre-norm 的优势表述成“去掉 warmup”，但放到今天更大的网络背景下，重点已经转移到一个更本质的问题：**训练稳定性与大规模学习率可用性**。

3.3 LayerNorm vs RMSNorm：现代模型为什么偏向后者

在 norm 类型上，第 3 讲同样给出了一条非常清晰的现实趋势：虽然原始 Transformer 使用 LayerNorm，但很多现代模型已经更偏向 RMSNorm。原因并不神秘。RMSNorm 不做去均值，也没有额外 bias 项，因此操作更简单、参数更少、数据搬运更轻；而在实践里，它常常又能达到与 LayerNorm 很接近的效果。

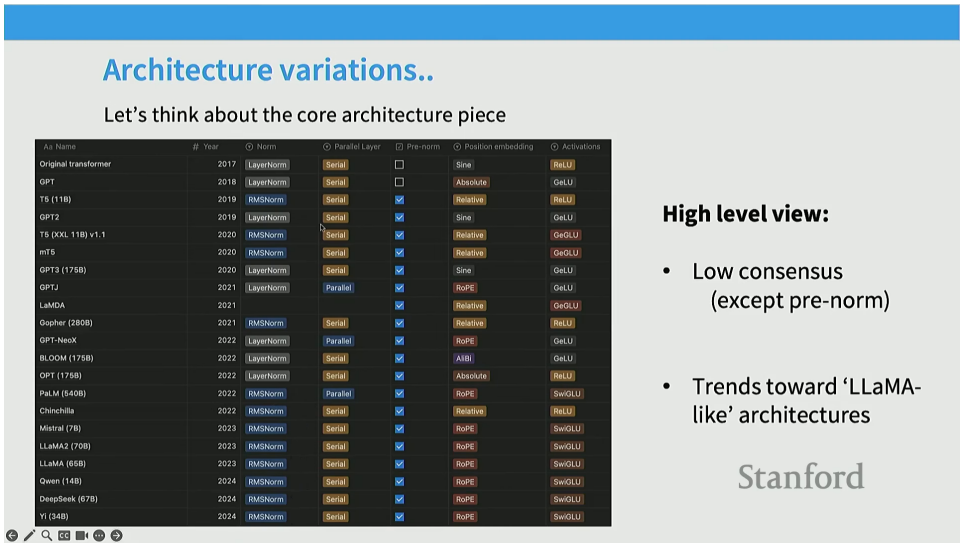

图 4: 课程把 LayerNorm 与 RMSNorm 放在同一页，强调现代模型更在意“够用且更省搬运”的 norm，而不是更复杂的规范化形式。[3]

[3] 视频画面时间区间：00:05:35–00:05:55。该图来自课程视频中的屏幕画面。

这一点和第 2 讲的资源会计语言正好接上：别只看 FLOPs，要看 runtime；别只看数学形式，要看 memory movement。RMSNorm 在这里就是一个非常典型的例子。

norm 选择最容易被误解的地方

课程没有说 RMSNorm 在所有场景都理论上更优，而是强调它在现代大模型里通常“够好而且更省”。这类判断的关键词是工程折中，而不是绝对优越。

3.4 更进一步：为什么现代模型还喜欢顺手去掉 bias

课程在 norm 讨论后顺手扩展到一个更普遍的趋势：线性层和 norm 层里的 bias 经常也被拿掉。原因与 RMSNorm 很像，并不是什么高深理论，而是因为这些项带来的参数和搬运开销，往往没有与之对等的明显收益。对大模型来说，这种“每层都省一点”的决策，累积起来会变得相当可观。

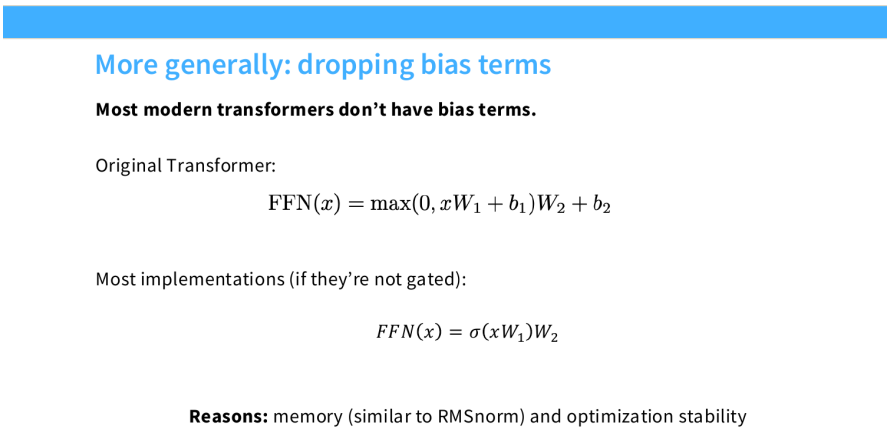

图 5: 课程把“dropping bias terms”单独拿出来讲，说明现代架构优化常常就是这种累计起来很重要的小决策。

3.5 本章小结

norm 与残差流这部分基本奠定了现代 LM 的底色：pre-norm 几乎成了事实标准；RMSNorm 是现代默认妥协；去 bias 化则是顺着同样的 runtime / memory 逻辑继续向前走的一步。

4 FFN 与激活函数：为什么现代模型偏爱 *GLU

4.1 ReLU、GeLU、SwiGLU 不是同一层面的选择

第 3 讲专门梳理了一大串激活函数与 FFN 变体：ReLU、GeLU、Swish、各种 *GLU。课程真正想表达的不是“看，有很多名字”，而是：**FFN 不只是线性层后面随便接个非线性，它是决定模型表达能力与计算配置的重要模块。**

原始 Transformer 时代，ReLU 非常自然；GPT 系列广泛使用 GeLU；而较新的模型又大量转向 SwiGLU / GeGLU。这个趋势说明，FFN 设计并不是边缘话题，它一直在被持续微调。

4.2 Gated linear units 为什么会被认为“通常更好”

课程解释 *GLU 家族的核心思想时非常直接：在 FFN 里增加一个逐元素门控通道，让激活不再只是“线性映射后过一个点函数”，而是“主激活 × 门控激活”的组合。这样做虽然会引入额外参数，但经验上往往会换来更好的性能。

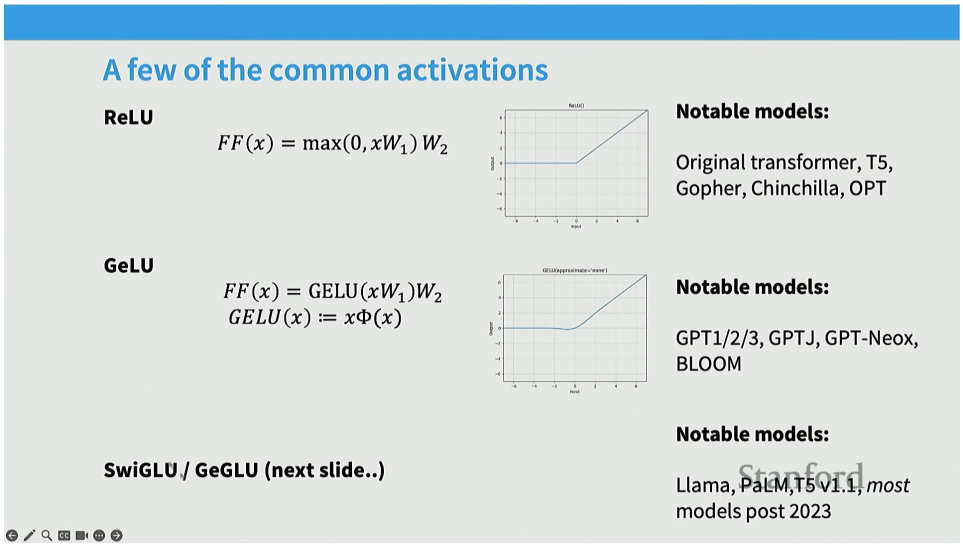

图 6: 课程直接展示 *GLU 系列相对普通 FFN 的效果表，结论很朴素：SwiGLU / GeGLU 通常值得认真考虑。[4]

课程给出的态度同样很成熟：*GLU 不是使用大模型的必要条件，GPT-3 这种系统不靠它也能很好；但如果你问“最近几年更常见、更被认可的默认选择是什么”，那答案大概率会指向 SwiGLU / GeGLU。

4.3 为什么 gated FFN 的维度比例会变

课程还提醒一个容易遗漏的工程细节：一旦 FFN 变成 gated 形式，很多模型会把传统的 $4\times d_{model}$ 规则缩到大约 $\frac{8}{3}\times d_{model}$ 附近。这不是随意魔改，而是因为 gated 结构本身已经增加了额外通道，不需要再照搬原始 ReLU FFN 的宽度比例。

关于激活函数选择的一个实用结论

如果你只是要选一个现代默认配置，SwiGLU 往往比 ReLU 更像“今天的大模型会怎么做”；但如果你要解释“为什么一定非它不可”，课程给出的答案是更谦虚的：大量经验表明它通常有效，而不是它在理论上被完全证明为唯一正确。

4.4 本章小结

FFN 与激活函数这一节的结论可以压缩成一句话：普通前馈层仍然可用，但 gated 变体尤其是 SwiGLU / GeGLU 已经在现代模型里形成了明显优势，因此成为更常见的默认路线。

[4] 视频画面时间区间：00:21:06–00:21:20。该图来自课程视频中的屏幕画面。

5 Position Embeddings、超参数与稳定性：哪些是共识，哪些只是经验盆地

5.1 位置编码为什么最终大多又收敛到 RoPE

课程在位置编码部分给出了一条很典型的“架构收敛史”：早期大家做过绝对位置编码、相对位置编码、ALiBi 等不同路线，但到了近几年，越来越多模型重新向 RoPE 收敛。原因在于 RoPE 提供了更自然的相对位置信号表达，同时能较好地嵌入 attention 计算过程里。

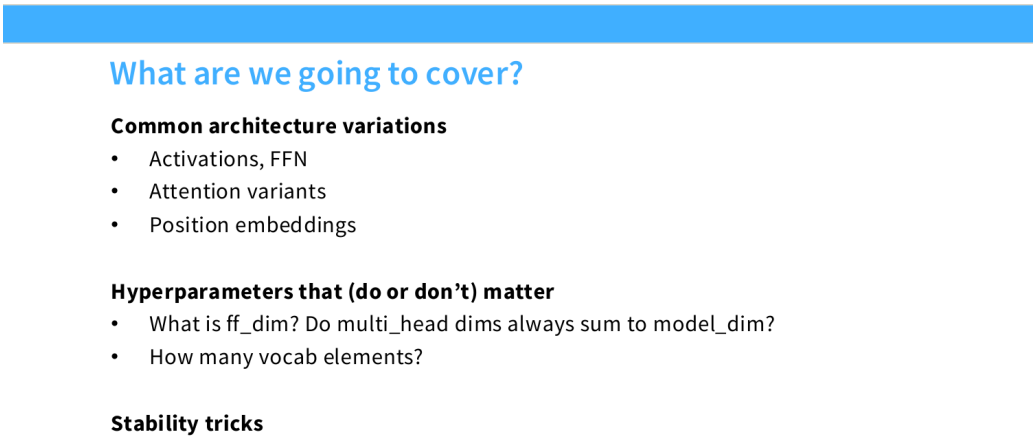

图 7: 课程用一页综述图把几类位置编码并列起来，最后把重点导向 RoPE。

课程对 RoPE 的解释也很有代表性：我们希望注意力机制尽量依赖相对位置信息，而旋转结构刚好能把这种相对关系嵌进内积几何里。这种解释比单纯背公式更有价值，因为它告诉你“RoPE 为什么看起来像今天的默认答案”。

5.2 Feedforward ratio、head dim、aspect ratio：为什么很多超参数 surprisingly boring

第 3 讲后半段的一个重要主题，是很多超参数其实比学生想象中更保守。比如：

- FFN 宽度相对 model dim 的比例，普通 FFN 常接近 4 倍，GLU 常接近 $\frac{8}{3}$ 倍。
- $num_heads \times head_dim \approx d_{model}$ 虽然不是数学定律，但在很多模型里近似成立。
- 模型是更深还是更宽，也有相对稳定的经验范围，而不是完全随意。

课程特别提到 T5 这类“大胆例外”，例如极高的 FFN 扩张倍率。这一类例外非常有用，因为它提醒我们：共识不等于定律；它只是说明在大多数训练环境里，一些范围已经被经验验证为“很难出大问题”的默认盆地。

关于超参数最值得记住的结论

现代大模型的大部分超参数并不花哨。真正稀缺的不是“想出一个奇怪设定”，而是知道什么时候该沿用保守共识，什么时候才值得偏离默认值。

5.3 regularization 与 weight decay：大模型真的还需要正则吗

课程在 regularization 部分刻意提出一个很有代表性的问题：既然预训练数据巨大、参数很多、往往只扫一遍数据，那还需要 regularization 吗？它给出的现实答案是：需要，但其作用更多体现在优化动态，而不只是传统意义上的防过拟合。

这也是为什么 dropout 在一些较新的模型里变少了，而 weight decay 仍然很常见。课程的处理方式非常工程化：**不要把正则化简单理解成“防过拟合开关”，而要把它看作训练动态调节器的一部分**。

5.4 稳定性技巧：z-loss、QK norm 与 soft-capping

到了 stability tricks 这部分，课程提醒读者一个常被低估的风险点：softmax 很容易在数值上变得难缠。于是近几年不少系统开始引入额外稳定性技巧，例如 output softmax 上的 z-loss、attention softmax 前的 QK norm，以及对 logits 做 soft-capping。

这些技巧的共同特点是：它们不是架构主干的一部分，却经常在超大规模训练里决定“曲线看起来是蓝色崩掉还是稳定下去”。这也是第 3 讲后半段最重要的隐含信息之一：**现代架构设计和稳定训练技巧之间已经很难完全分家**。

5.5 GQA / MQA：为什么这讲最后会碰到 attention head 成本

课程在结尾还专门把 GQA / MQA 拿出来讲，原因不是它们只是“注意力的小变体”，而是因为它们直接连到推理成本。训练阶段看起来 attention 还比较容易并行，但到了增量生成阶段，KV cache 让 head 数量和 key/value 维度变得极其重要。于是，减少 key/value head 数量就会显著影响 inference memory cost。

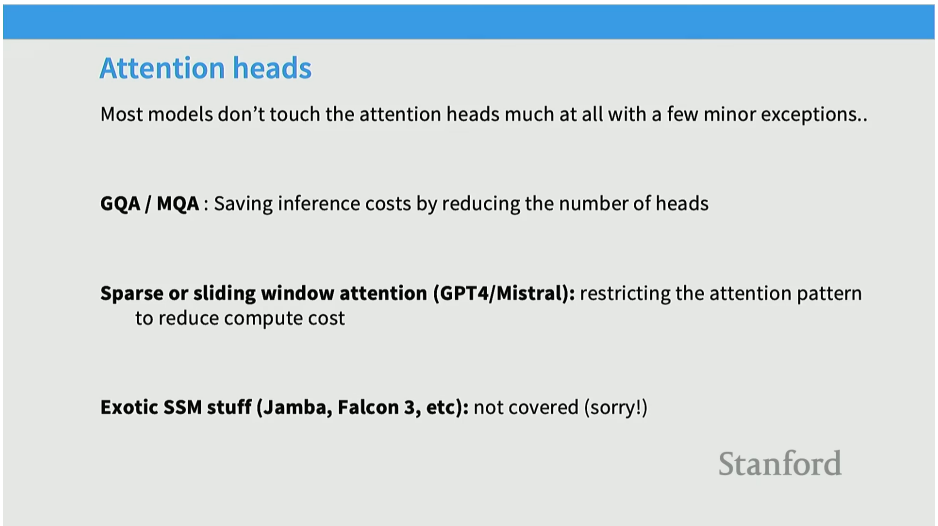

图 8: 第 3 讲把 GQA / MQA 放到结尾，是为了提前把“架构选择会改写推理成本”这个思想种下去。[5]

这也正是 CS336 的典型风格：讲架构，从来不是只讲表达能力，而是同时讲训练、推理与系统代价。

[5] 视频画面时间区间：01:15:47–01:16:10。该图来自课程视频中的屏幕画面。

5.6 本章小结

位置编码、超参数、regularization 与 GQA/MQA 这些看似分散的话题，实际上都指向同一个问题：哪些设计是已经足够稳定、值得默认采用的；哪些设计虽然可能有效，但仍需要你在具体资源与任务条件下重新判断。

6 总结与延伸

第 3 讲最重要的贡献，是把“现代大模型架构经验”从零散的模型传闻，整理成一套较为稳定的工程共识。

如果只抓住五个结论，我会这样概括：

1. 现代 Transformer 整体上越来越向 LLaMA-like 方向收敛。
2. pre-norm 基本已经成为事实标准，RMSNorm 也已非常常见。
3. SwiGLU / GeGLU 代表了 FFN 的现代默认升级路径。
4. RoPE 已经在位置编码里形成了强收敛趋势。
5. 大多数超参数其实比你想象得保守，真正困难的是知道何时偏离默认共识。

我对第 3 讲的压缩结论

现代大模型架构并不是一团随机试错后的混沌集合。它已经形成了一片相对稳定的经验盆地，而第 3 讲的任务，就是教你识别这片盆地的边界与内部共识。

对后续章节来说，这一讲有两个直接延伸方向。第一，架构共识会决定后面 kernel、parallelism 和 inference 的成本结构。第二，看似“只是训练小技巧”的稳定性方法，实际上会在大规模训练里变得和主架构一样重要。也就是说，从这一讲开始，架构设计就不再只是模型层问题，而是完整系统设计的一部分。